

COMP719 - OPTIMIZATION AND MODELLING
SEMESTER PROJECT
PROJECT TITLE: THE CAMPUS LAB SPACE
OPTIMIZER PHASE: PHASE 2 - GREEDY HEURISTIC
IMPLEMENTATION

1. GROUP INFORMATION

Group Members and Contribution Estimates:

Member Name	Student ID	Role	% Contribution
Jeryn Naidoo	223009245	Performance Analysis & Feasibility Validation	20
Kirone Gopaul	223014456	Implementation Lead & Data Preparation	20
Shaista Naicker	222106172	Algorithm Design & Phase 3 Planning	20
Josh Goodwin	222092144	Testing, Results Verification & Output Validation	20
Tashmika Pillay	223097300	System Overview, Documentation & Report Compilation	20

2. CURRENT SYSTEM OVERVIEW

2.1 What Your Program Can Do Now

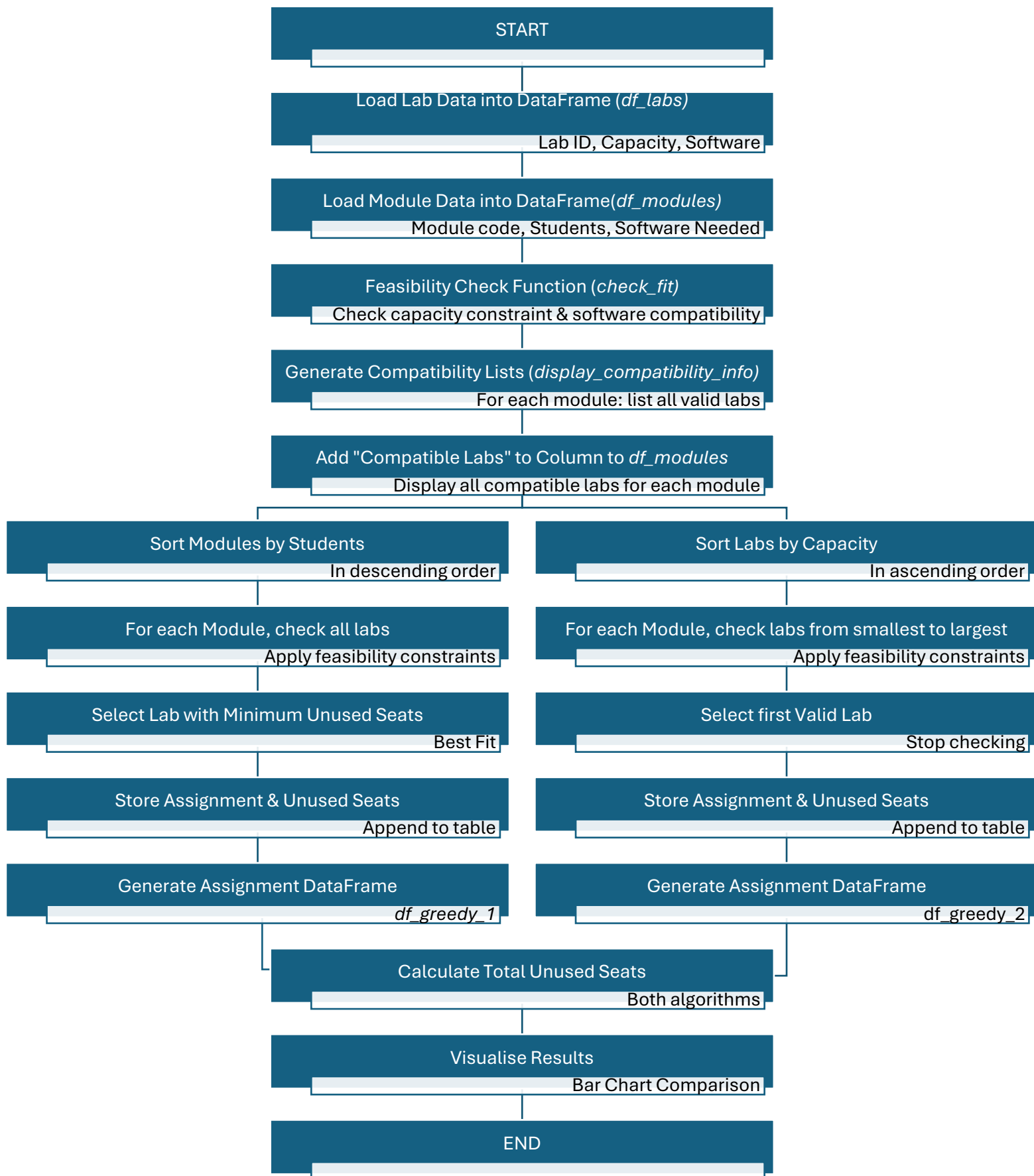
The current implementation of our Campus Lab Space Optimizer provides a complete framework to generate feasible lab allocations, making use of greedy heuristic strategies. Our program begins by loading structured lab and module data into Pandas DataFrames, which allows for easier manipulation and evaluation of the dataset.

Additionally, the system includes a feasibility checking mechanism that ensures all assignments satisfy both the capacity and software constraints. This is done through a dedicated function that evaluates whether a given module can be allocated to a specific lab without violating these constraints. The program makes use of this function to construct a compatibility mapping that identifies all valid lab options for each module.

Furthermore, the program implements two distinct greedy algorithms to generate complete assignment solutions. The first algorithm prioritizes modules with the largest number of enrolments and assigns them to the most space-efficient compatible lab. The second algorithm processes modules in their original order and assigns each to the smallest available lab that ensures its constraints are not violated. Both algorithms guarantee that every module is assigned to only one valid lab.

For each solution, the program then calculates the number of unused seats per assignment and aggregates these values to determine the total seat wastage. The results are shown in tabular form and further visualized using a bar chart to make accurate comparisons between the two greedy strategies.

2.2 Program Flow



3. GREEDY ALGORITHM DESIGN

3.1 Greedy Rule Used

The greedy approach works by making the best available decision at each step without going back to reconsidering earlier ones. It does not explore all possibilities. It commits, moves on, and builds a solution one assignment at a time. That makes it fast and easy to reason, which is exactly why it is a useful starting point before applying more powerful techniques. We implemented two greedy strategies and compared them.

Greedy Algorithm 1 - Largest Module First (Best-Fit Decreasing):

Before any assignments are made, modules are sorted in descending order of student count. For each module, the algorithm then scans all labs and selects the one that wastes the fewest seats, meaning the smallest compatible lab that still fits. This mirrors the Best-Fit Decreasing strategy from classical bin-packing theory.

The logic is straightforward. Larger modules have fewer labs that can accommodate them. M1 has 85 students and needs Python, so only L1 fits. M7 has 90 students and needs Java, and again only L1 fits. If those modules are not handled first, a smaller module could claim L1 and leave the bigger ones with nowhere to go. Sorting by size ensures the hardest-to-place modules get priority before options run out. Selecting the least-wasteful lab at each step then directly targets the objective function.

Greedy Algorithm 2 - Smallest Lab First:

Modules are processed in their original order (M1 through M10). For each module, labs are sorted from smallest to largest capacity, and the first one that fits is selected with no further checking. There is no sorting by module size and no consideration of how constrained a module is. It just works through the list and always prefers the smallest compatible lab available.

The idea is simple. If a small lab fits, use it. Do not put a small class in a large room when a smaller one will do. It does not account for which modules are harder to place, and that is the key difference from Algorithm 1. One is smarter about ordering, the other is smarter about selection at each step.

3.2 Pseudocode

Greedy Algorithm 1 - Largest Module First:

```
FUNCTION greedy_largest_first():
    sorted_modules ← sort df_modules by Students DESCENDING
    result ← empty list

    FOR each module in sorted_modules:
        best_lab ← None
        least_waste ← ∞

        FOR each lab in df_labs:
            IF lab.Software contains module.SoftwareNeeded
               AND module.Students ≤ lab.Capacity THEN:
                waste ← lab.Capacity - module.Students
            IF waste < least_waste THEN:
                least_waste ← waste
                best_lab ← lab

        APPEND (module, best_lab, least_waste) TO result

    RETURN result as DataFrame
```

Greedy Algorithm 2 - Smallest Lab First:

```
FUNCTION greedy_smallest_lab_first():
    sorted_labs ← sort df_labs by Capacity ASCENDING
    result ← empty list

    FOR each module in df_modules (original order):
        assigned_lab ← None
```

```

FOR each lab in sorted_labs:
    IF lab.Software contains module.SoftwareNeeded
    AND module.Students ≤ lab.Capacity THEN:
        assigned_lab ← lab
        BREAK // take the first valid lab

waste ← assigned_lab.Capacity - module.Students
APPEND (module, assigned_lab, waste) TO result

RETURN result as DataFrame

```

3.3 Feasibility Handling

Feasibility is enforced at every assignment decision through the `check_fit()` function, which evaluates two conditions in parallel. First, the capacity constraint: the module's student count must not exceed the lab's seating capacity. Second, software compatibility: the lab must have the required software installed. The function only returns True when both conditions are satisfied simultaneously. This means neither algorithm can ever assign a module to an incompatible lab. Infeasible assignments are structurally impossible within the code. It is also worth noting that labs are permitted to host multiple modules across the semester, since all modules are scheduled at different times. This is an intentional modelling assumption, not an oversight.

4. IMPLEMENTATION DETAILS

4.1 Data Structures Used

The primary data structure used throughout the project remains a Panda's dataframe. Each module is assigned to an individual row, which serves as a unique identifier for that row. Subsequent columns store module attributes such as: Number of students, Lab ID for the assigned lab, Capacity and Number of Unused Seats. However, when assigning labs

and the number of unused seats, each row is stored in a simple Python list and converted to a Panda's dataframe for easier use, manipulation, and visualisation.

4.2 Key Functions / Modules

Function Name	Purpose
<code>greedy_largest_first()</code>	Implements a greedy algorithm to assign modules to labs. It prioritises assigning modules with the largest number of students first. For each module, it attempts to find the lab that best fits its requirements (software and capacity) while minimising the number of unused seats.
<code>greedy_smallest_lab_first()</code>	implements another greedy algorithm for assigning modules to labs. This strategy focuses on trying to fit modules into the smallest available labs first

5. RESULTS FROM GREEDY ALGORITHM

5.1 Final Assignment Output

<u>Greedy Largest First</u>					
	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M7	90	L1	100	10
1	M1	85	L1	100	15
2	M4	55	L2	60	5
3	M10	50	L2	60	10
4	M2	45	L1	100	55
5	M5	35	L3	40	5
6	M8	30	L3	40	10
7	M3	25	L4	30	5

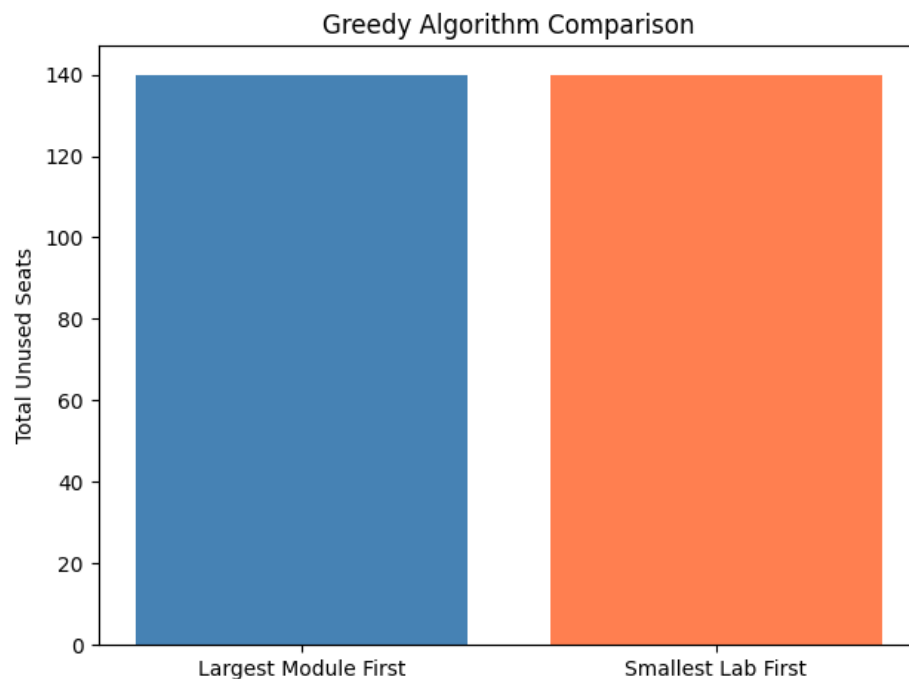
8	M9	20	L4	30	10
9	M6	15	L4	30	15

Greedy Smallest First

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M1	85	L1	100	15
1	M2	45	L1	100	55
2	M3	25	L4	30	5
3	M4	55	L2	60	5
4	M5	35	L3	40	5
5	M6	15	L4	30	15
6	M7	90	L1	100	10
7	M8	30	L3	40	10
8	M9	20	L4	30	10
9	M10	50	L2	60	10

5.2 Total Unused Seats

After a comparison of two greedy solutions we have implemented, the results concluded that both solutions returned the same number of unused seats.



5.3 Solution Validity

To ensure every module is assigned exactly once, we can view the following results:

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M7	90	L1	100	10
1	M1	85	L1	100	15
2	M4	55	L2	60	5
3	M10	50	L2	60	10
4	M2	45	L1	100	55
5	M5	35	L3	40	5
6	M8	30	L3	40	10
7	M3	25	L4	30	5
8	M9	20	L4	30	10
9	M6	15	L4	30	15

As a result of the greedy algorithm, it is evident that no module, in the final assignment matrix, is repeated. A similar assignment matrix exists for the smallest lab first method.

To satisfy every constraint, the following code is used:

```
# Does this lab have the right software?
software_ok = module['Software Needed'] in lab['Software']

# Does this lab have enough seats?
space_ok = module['Students'] <= lab['Capacity']

if software_ok and space_ok:
    wasted_seats = lab['Capacity'] - module['Students']
```

This code makes use of Boolean values to check the required constraints. The Boolean values are then used to validate the data before performing any calculations on the number of wasted seats.

6. PERFORMANCE DISCUSSION

6.1 Strengths of Your Greedy Approach

The general benefits of making use of a greedy algorithm are that they are quick and simple to implement. It allows for a rapid development environment, whilst also being easy to comprehend and build upon in the future of the project's life cycle. In reference to our project, the greedy algorithms allow us to handle constraints easily and provide a baseline model for us to build upon in the future with non-greedy algorithms such as Hill-Climbing or Swap-Search.

6.2 Limitations of Your Greedy Approach

The implemented greedy algorithms fall victim to the typical issues with greedy solutions. Whilst they are easy to implement and develop, they lack the precision of a non-greedy solution. Our greedy approaches both provide solutions rapidly; neither solution is optimal since they both get stuck in local optima, due to the lack of backtracking, and are both dependent on the ordering of the data.

7. PREPARATION FOR PHASE 3 (IMPROVEMENT STRATEGY)

7.1 Planned Local Search Idea

The greedy algorithms give us a valid, feasible starting point but not necessarily the best one. The nature of greedy decision-making is that each step looks locally optimal without any awareness of how it affects the choices that come later. That is why Phase 3 exists: to take what the greedy produced and ask whether any small change can make it better. Two improvement methods are planned.

The first is hill-climbing local search. We look at the current solution, try a small change, and if that change reduces total unused seats, we keep it. This repeats until no improvement can be found, at which point the solution has reached a local optimum. The search terminates when a full pass through all modules produces no reduction in total unused seats.

The second is swap search. Rather than moving one module at a time, this method takes two modules and exchanges their lab assignments. If both modules remain in valid labs after the swap and the combined unused seats decrease, the swap is accepted.

7.2 Neighbourhood Definition (Simple Description)

A neighbourhood move is any small change to the current assignment that produces a new valid assignment. Two types are defined.

Reassignment move: Take one module and try moving it to a different compatible lab. If the new lab has a smaller capacity gap than the current one, the move is accepted and everything else stays the same. The plan is to implement a reassignment function that loops through every module, checks every other valid lab, and commits to the first improvement it finds. It then restarts and keeps going until a full pass produces nothing better.

Swap move: Take two modules and exchange their lab assignments. For a swap to be accepted, both modules must remain in valid labs after the exchange, and the combined unused seats must decrease. The plan is to implement a swap function that tries every pair of modules, checks feasibility in both directions, and only commits if the total waste goes down.

Hill-climbing can get stuck when improving one module requires first moving another out of the way, which is exactly the kind of improvement swap search is designed to find. Both methods keep all generated solutions feasible, so no constraint is ever at risk.

8. CHALLENGES ENCOUNTERED

One of the challenges encountered was ensuring feasibility at every step. Initially, there was a potential risk of assigning modules to incompatible labs, so it was resolved by implementing strict checks for both capacity and software in the *check_fit()* function and implementing them into both greedy algorithms.

Another challenge we faced was designing effective greedy strategies that balance feasibility and efficiency. Assigning modules based on size seems to be intuitive, however, it required careful justification to ensure it would not inefficiently impact the overall performance. To counter this, we implemented two different greedy strategies and compared them to ensure the results were reliable.

However, it was still a challenge comparing the effectiveness of the different greedy approaches and determining which produced a better solution, as both algorithms generate valid assignments but follow different decision-making approaches. Simply observing individual assignments was not sufficient to evaluate their performance. To address this, we calculated the total number of unused seats as a quantitative comparison metric. A visual comparison was further implemented, which allowed for an objective evaluation of the approaches.

9. CURRENT CODE STATUS

The current implementation successfully meets the objectives of Phase 2. The program loads and structures the dataset, performs feasibility checks, and generates valid assignments using the two greedy strategies. Each module is assigned to only one compatible lab, and the constraints are strictly enforced. Furthermore, the calculation of total unused seats and the visual comparison of the two algorithms successfully allow for an evaluation of efficiency.

However, the solution is limited by the nature of greedy algorithms themselves. The assignments are feasible but do not guarantee to be optimal. Decisions are sequentially made without reconsideration so earlier choices may lead to unnecessary seat wastage in the final solution. To address this, the next phase will focus on improving the greedy solution by making use of local search techniques. Using the greedy approaches' outputs as a basis, small changes will be explored to reduce the unused seats. One approach brainstormed involves reassigning a module to a different compatible lab if it is a better fit. Another approach contemplates swapping the lab assignments of two modules, allowing the algorithm to explore the improvements that cannot be made through single moves alone.

These changes will follow a hill-climbing strategy where changes are accepted only if they reduce the total unused seats. A locally optimal solution will be achieved when no further improvements can be found, ensuring the final solution is valid, enforcing feasibility constraints, and efficient.

10. SCREENSHOTS OR CODE SNIPPETS (OPTIONAL)

```
def greedy_largest_first():
    # Sort the modules so the biggest ones (most students) go first
    sorted_modules = df_modules.sort_values('Students', ascending=False)
    sorted_modules = sorted_modules.reset_index(drop=True)

    result_rows = []

    # Go through every module one at a time
    for i in range(len(sorted_modules)):
        module = sorted_modules.iloc[i]

        best_lab = None
        least_waste = 999999 # Start with a big number so any real waste will be smaller

        # Check every lab to find the one that wastes the fewest seats
        for j in range(len(df_labs)):
            lab = df_labs.iloc[j]

            # Does this lab have the right software?
            software_ok = module['Software Needed'] in lab['Software']

            # Does this lab have enough seats?
            space_ok = module['Students'] <= lab['Capacity']

            if software_ok and space_ok:
                wasted_seats = lab['Capacity'] - module['Students']

                # If this lab wastes fewer seats than the current best, use it instead
                if wasted_seats < least_waste:
                    least_waste = wasted_seats
                    best_lab = lab

        if best_lab is None:
            raise ValueError(f"No feasible lab found for module {module['Module Code']}")

        # Save the result for this module
        row = {
            'Module Code': module['Module Code'],
            'Students': module['Students'],
            'Lab ID': best_lab['Lab ID'],
            'Capacity': best_lab['Capacity'],
            'Unused Seats': least_waste
        }
        result_rows.append(row)

    return pd.DataFrame(result_rows)
```

Figure 1 Function implementing greedy algorithm 1 : Largest Module First

```

def greedy_smallest_lab_first():
    # Sort the labs so the smallest ones are tried first
    sorted_labs = df_labs.sort_values('Capacity')
    sorted_labs = sorted_labs.reset_index(drop=True)

    result_rows = []

    # Go through every module one at a time
    for i in range(len(df_modules)):
        module = df_modules.iloc[i]

        assigned_lab = None

        # Try each lab from smallest to largest – pick the first one that fits
        for j in range(len(sorted_labs)):
            lab = sorted_labs.iloc[j]

            # Does this lab have the right software?
            software_ok = module['Software Needed'] in lab['Software']

            # Does this lab have enough seats?
            space_ok = module['Students'] <= lab['Capacity']

            if software_ok and space_ok:
                assigned_lab = lab
                break # Found a fitting lab, no need to check any more

        if assigned_lab is None:
            raise ValueError(f"No feasible lab found for module {module['Module Code']}")

        # Save the result for this module
        unused_seats = assigned_lab['Capacity'] - module['Students']
        row = {
            'Module Code': module['Module Code'],
            'Students': module['Students'],
            'Lab ID': assigned_lab['Lab ID'],
            'Capacity': assigned_lab['Capacity'],
            'Unused Seats': unused_seats
        }
        result_rows.append(row)

    return pd.DataFrame(result_rows)

```

Figure 2 Function implementing greedy algorithm 2: Smallest Lab First

df_greedy_1 = greedy_largest_first() df_greedy_1					
	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M7	90	L1	100	10
1	M1	85	L1	100	15
2	M4	55	L2	60	5
3	M10	50	L2	60	10
4	M2	45	L1	100	55
5	M5	35	L3	40	5
6	M8	30	L3	40	10
7	M3	25	L4	30	5
8	M9	20	L4	30	10
9	M6	15	L4	30	15

df_greedy_2 = greedy_smallest_lab_first() df_greedy_2					
	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M1	85	L1	100	15
1	M2	45	L1	100	55
2	M3	25	L4	30	5
3	M4	55	L2	60	5
4	M5	35	L3	40	5
5	M6	15	L4	30	15
6	M7	90	L1	100	10
7	M8	30	L3	40	10
8	M9	20	L4	30	10
9	M10	50	L2	60	10

Figure 3 Output results of both greedy algorithms

```


labels = ['Largest Module First', 'Smallest Lab First']
totals = [df_greedy_1['Unused Seats'].sum(), df_greedy_2['Unused Seats'].sum()]

fig, ax = plt.subplots()
ax.bar(labels, totals, color=['steelblue', 'coral'])
ax.set_ylabel('Total Unused Seats')
ax.set_title('Greedy Algorithm Comparison')
plt.tight_layout()
plt.show()


```

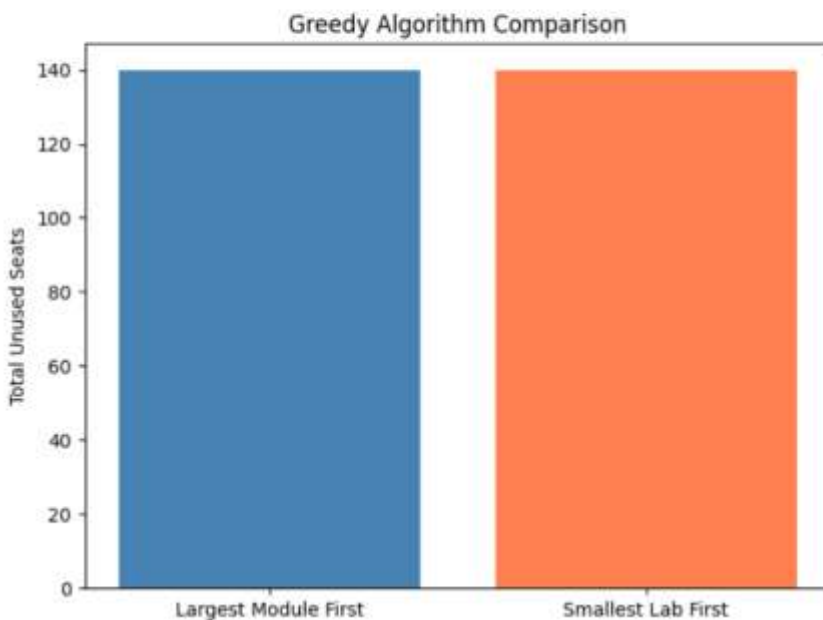


Figure 4 Greedy algorithms Visual Comparison using Bar Chart

11. DECLARATION

We confirm that:

- This work is our own.
- All group members contributed as stated.

Signed:

- ✕ Jeryn Naidoo
- ✕ Shaista Naicker
- ✕ Kirone Gopaul
- ✕ Josh Goodwin
- ✕ Tashmika Pillay

Date:

16/04/2026